

SD3589CV100

AB分区启动与升级用户指南

文档版本　1.0.3

发布日期　2025-05-28

前 言

概述

本文档主要介绍 SD3589CV100芯片按照AB分区启动和升级方案的设计说明，旨在指导工程师如何快速使用本产品并进行二次开发。

版本说明

与本文档相对应的产品版本如下。

产品名称	产品版本
SD3589C	V100
SD3589D	V100

用户

本文档主要适用于以下工程师：

- ✧ 技术支持工程师
- ✧ 软件工程师
- ✧ 算法工程师

修订记录

修订记录列出了各文档版本间发生的主要更改。最新版本的文档包含以前所有文档版本的更新内容。

版本	修订日期	修订说明
v1.0.0	2024.04.17	首个OTA使用参考文档
v1.0.1	2024.09.27	增加镜像编译介绍部分
V1.0.2	2024.10.10	增加安全校验相关功能
V1.0.3	2025.05.28	优化sample流程图和增加检验与解密流程

版权所有 © 上海云集互联有限公司

目 录

目录

前 言.....I
目 录.....II
插图目录.....III
表格目录.....IV
1. 概述.....1
1.1. 镜像介绍.....1
1.1.1. 镜像编译.....1
1.1.2. 镜像构成.....1
1.2. 启动流程介绍.....2
1.3. 分区结构介绍.....3
1.4. 升级流程介绍.....3
2. 方案说明.....5
2.1. 分区管理(Partition.bin).....5
2.1.1. 分区配置文件的构建.....5
2.1.2. 分区配置文件字段说明及实例.....5
2.2. 相关块设备挂载情况.....6
2.3. BootCtrl API.....6
2.4. 防回滚机制.....7
2.5. 检验与解密.....7
2.6. Upgrade Sample.....8
2.6.1. 升级包的构建.....8
2.6.2. 基于AB分区升级流程.....8

版权所有 © 上海云集互联有限公司

插图目录

图 1-1 以spi非快启非安全启动为例编译后镜像示例图..... 1
图 1-2 AB分区启动流程图..... 2
图 1-3启动介质分区结构图..... 3
图 1-4 升级流程状态转换图..... 3
图 2-1 分区管理部署图..... 5
图 2-2 partition.xml文件实例..... 5
图 2-3 相关块设备挂载情况spi nand单独生成升级包流程图..... 6
图 2-4 升级时BooCtrl 修改A/B slot的启动状态信息API调用流程..... 7
图 2-5校验升级固件流程图..... 7
图 2-6 spi nand单独生成升级包流程图..... 8
图 2-7 升级sample程序的工作流程..... 9

版权所有 © 上海云集互联有限公司

表格目录

表 1-1 不同镜像介绍.....1
表 2-1 partition_info 字段说明.....6

版权所有 © 上海云集互联有限公司

1. 概述

1.1. 镜像介绍

1.1.1. 镜像编译

SD3589CV100默认启动方式为单分区启动，如需使用AB分区启动方案需要在lq560v100_sdk/main/src/platform路径下将Makefile文件中

第55行，BOOT_MODE ?= normal注释掉，

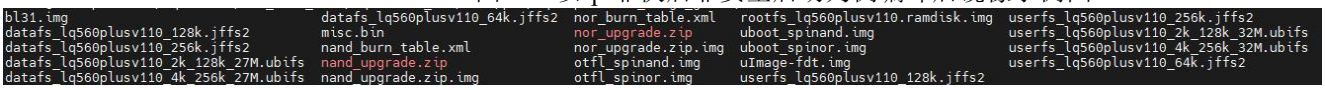
第56行，BOOT_MODE ?= ab放开注释。

注意：在更改启动方式时需先对SDK进行clean处理。

1.1.2. 镜像构成

以SPI-Flash 非快启非安全启动镜像为例，SDK代码编译后默认得到的镜像文件(路径为： main/pub/lq560plusv10_image_glibc)，如下图1-1所示。

图 1-1 以spi非快启非安全启动为例编译后镜像示例图



不同的镜像与分区文件中的分区名称字段对应可参考下表1-1：

表 1-1 不同镜像介绍

名称	分区名称	说明
bl31.img	bl31_a/bl31_b	/
misc.bin	misc	/
nand_burn_table.xml nor_burn_table.xml	/	分区配置文件[1]
nand_upgrade.zip nor_upgrade.zip	/	原始压缩包
nand_upgrade.zip.img nor_upgrade.zip.img	/	升级文件[1]
otfl_spinand.img otfl_spinor.img	otfl	一级启动loader[1]
uboot_spinand.img uboot_spinor.img	uboot_a/ uboot_b	U-Boot镜像[1]
ulmage-fdt.img	kernel_a/ kernel_b	Kernel镜像
rootfs_lq560plusv100.ramdisk.img	rootfs_a/rootfs_b	Rootfs镜像
userfs_lq560plusv100_2k_128k_32M.ubifs userfs_lq560plusv100_4k_256k_32M.ubifs userfs_lq560plusv100_64k.jiffs2 userfs_lq560plusv100_128k.jiffs2 userfs_lq560plusv100_256k.jiffs2	userfs_a/ userfs_b	UserFS镜像[2]
datafs_lq560plusv100_4k_128_27M.ubifs	data	DataFS镜像[2]

版权所有 © 上海云集互联有限公司

名称	分区名称	说明
datafs_lq560plusv100_4k_256_27M.ubifs datafs_lq560plusv100_64k.jiffs2 datafs_lq560plusv100_128k.jiffs2 datafs_lq560plusv100_256k.jiffs2		

[1]此镜像需根据启动介质(spi-nand/spi-nor)选择对应镜像。

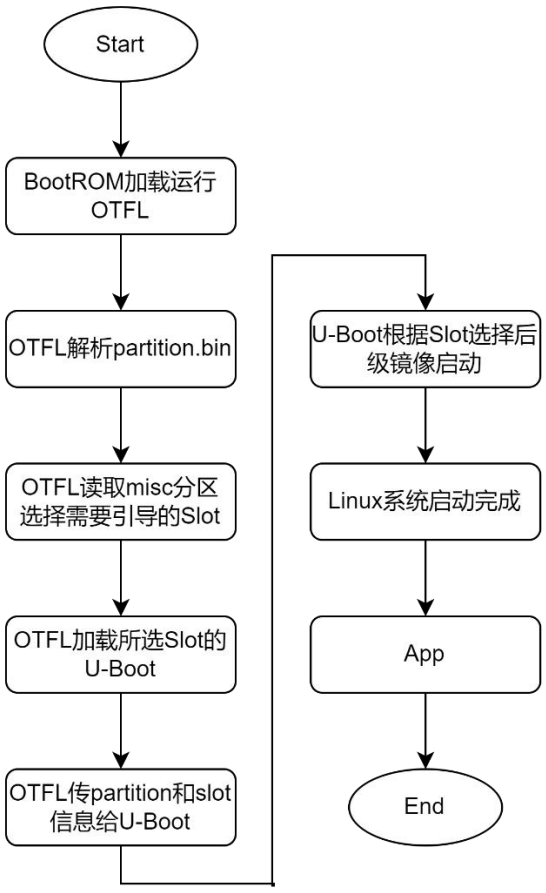
[2]ubifs格式对应启动介质为spi-nand，文件名中区分了不同的spi-nand参数，比如其中的2k_128k_32M表示页大小为2KB，block大小为128KB，分区大小32MB；jiffs格式对应启动介质为spi-nor，文件名中区分了不同给的spi-nor参数，比如其中的64k表示block大小为64KB；用户必须根据具体的启动介质参数选择不同镜像文件。

注意：镜像后缀带有“.sig”的镜像是安全启动镜像，详细介绍请参考《SD4589CV100 安全启动开发指南.pdf》

1.2. 启动流程介绍

SD3589CV100 芯片AB分区启动流程如下图1-2所示。

图 1-2 AB分区启动流程图



其具体步骤为：

1. BootROM加载OTFL镜像；
2. OTFL解析partition.bin；
3. OTFL通过读取msic分区信息确定当前是从A slot还是B slot区域启动；
4. OTFL加载所选slot的U-Boot；
5. OTFL将partition和slot信息传给U-Boot；
6. U-Boot根据slot选择后级启动镜像；

版权所有 © 上海云集互联有限公司

7. Linux系统启动完成。

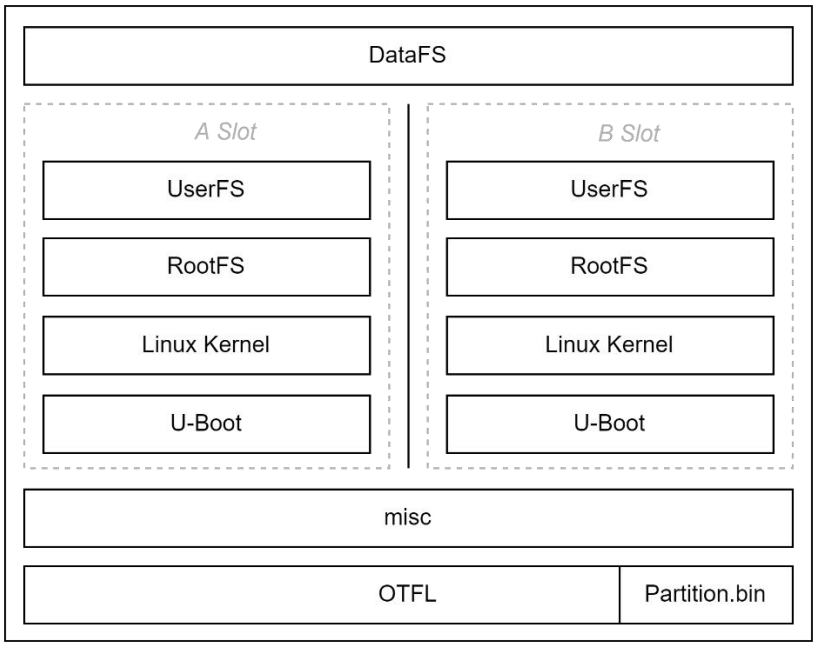
1.3. 分区结构介绍

启动介质分区结构如图1-3所示。

在启动过程中，OTFL会读取Partition.bin文件解析分区信息以及读取misc分区选择当前从A slot分区还是B slot分区启动。

A/B分区相互独立，分别包含U-Boot、Linux Kernel、RootFS、UserFS文件。其中，RootFS是Linux系统的只读根文件系统；UserFS是用户二次开发时部署应用的分区，该分区挂载在Linux系统根文件系统的/opt目录下；DataFS分区只有一份，用于存放用户应用运行时所产生的数据，该分区挂载在Linux系统跟文件系统的/data目录下。正常情况下，OTFL、misc、DataFS为出厂时烧写好的分区，在设备进行AB分区升级时不参与升级。

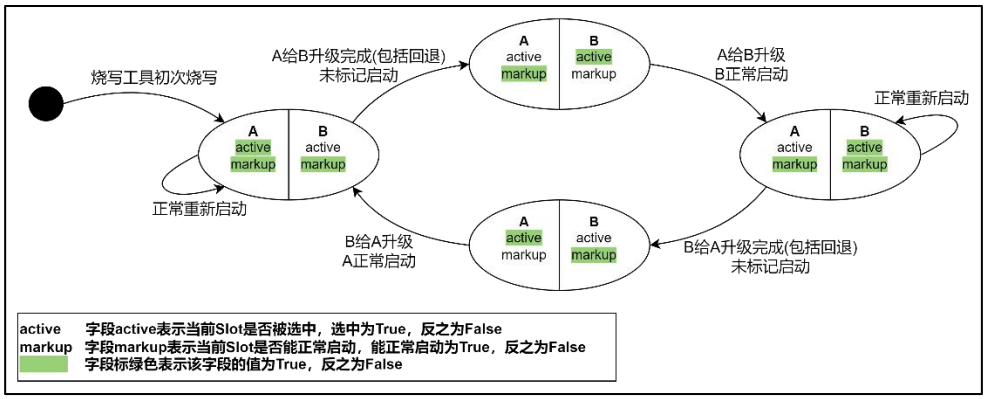
图 1-3启动介质分区结构图



1.4. 升级流程介绍

升级流程的状态转换如下图1-4所示。

图 1-4 升级流程状态转换图



图中：active表示A/B Slot中当前slot是否被选中，如果当前slot被选中则对应active为True，否则为False；markup表示当前Slot是否能正常启动，如果当前slot能正常启动则

版权所有 © 上海云集互联有限公司

markup为True，否则为False。

其升级流程过程如下：

1. 空板第一次烧写完成后，默认A和B两个slot都是可以正常启动的，但优先启动A slot；

2. 当通过A slot给B slot升级或者从A slot主动切换到B slot时，B slot的active变为True，但是其markup暂时为False；

3. 当B slot在升级后第一次启动成功时，B slot的markup被标记为True；此后正常重新启动时都从B slot启动；

4. 当通过B slot给A slot升级或者从B slot主动切换到A slot时，A slot的active变为True，但是其markup暂时为False；

5. 当A slot在升级后第一次启动成功时，A slot的markup被标记为True；此后正常重新启动时都从A slot启动；此时固件状态回到了第一次烧写时的状态。

版权所有 © 上海云集互联有限公司

2. 方案说明

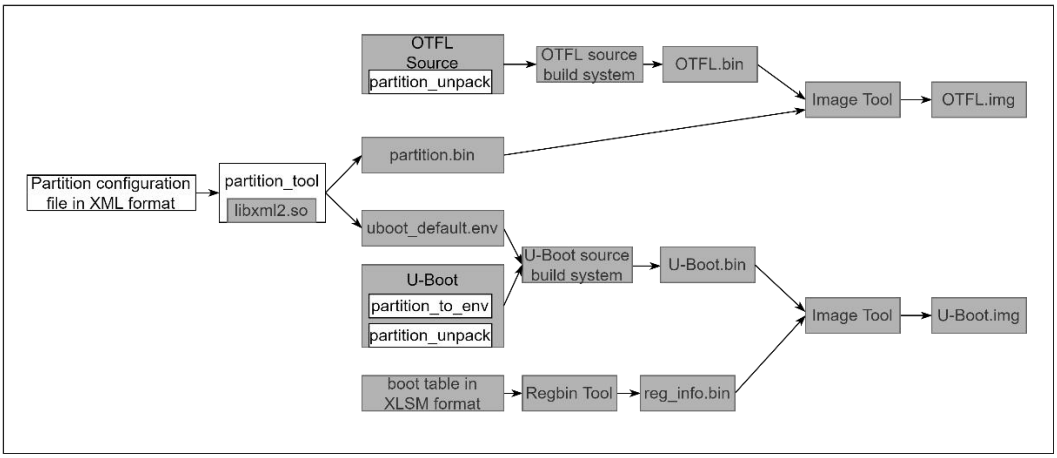
2.1. 分区管理(Partition.bin)

2.1.1. 分区配置文件的构建

SDK提供了xml格式的分区文件，该分区文件中可以配置分区信息，同时也可以增加U-Boot中的默认环境变量。编译时与启动该介质相关的xml文件将被编译成partition.bin和uboot_default.env文件，前者会被包含到OTFL.bin文件中，后者则会放入U-Boot的源码中编译成U-Boot的默认环境变量。

xml路径为：main/tools/pc/partition_env/config/lq560plusv100。

图 2-1 分区管理部署图



2.1.2. 分区配置文件字段说明及实例

图2-2提供了一份以spi-nand为启动介质的partition.xml文件实例。

图 2-2 partition.xml文件实例

```
<?xml version="1.0" encoding="UTF-8" ?>
<boot_media_info boot_media_infos ="">
        <partition_info boot_media_partitions_info="">
                <partition  partition_name="partition" path="partition.img" flash_type="nand" fs_type="none" start_addr="0"    length="512k" />
                <partition  partition_name="otfl"      path="otfl.img"      flash_type="nand" fs_type="none" start_addr="512k" length="512k" />
                <partition  partition_name="otsl"      path="otsl.img"      flash_type="nand" fs_type="none" start_addr="1M"   length="1M" />
                <partition  partition_name="uboot"     path="uboot.img"     flash_type="nand" fs_type="none" start_addr="2M"   length="2M" />
                <partition  partition_name="kernel"    path="kernel.img"    flash_type="nand" fs_type="none" start_addr="4M"   length="2M" />
                <partition  partition_name="rootfs"    path="rootfs.img"    flash_type="nand" fs_type="ext4" start_addr="6M"   length="20M" />
        </partition_info>
        <env_info boot_media_env_info="">
                 <env  enable="1" env="netdev=eth0\0
                                       consoledev=ttyS1\0
                                       ramdiskaddr=600000\0
                                       ramdiskfile=your.ramdisk.u-boot\0
                                       fdtaddr=400000\0 fdtfile=your.fdt.dtb\0"  />
        </env_info>
</boot_media_info>
```

<boot_media_partitions_info>字段填写的是分区信息，<boot_media_env_info>字段填写的是U-Boot中默认的环境变量（所有环境变量用一个长字符串表示，相邻两个变量之间使用“\0”隔开）。目前预留给partition.bin的空间大小为8kB，用户可根据业务按需更改partition.xml包含的partition_info及env_info部分，但要注意编译产生的partition.bin不要超过8KB大小。

分区配置文件中相关字段含义及备注信息如下表2-1所示。注意Flash类型字段中填写的nand和nor分别指spi-nand和spi-nor。用户可以借用烧写工具的的分区烧写界面编辑

版权所有 © 上海云集互联有限公司

保存该分区信息字段的内容。

表 2-1 partition_info 字段说明

字段名称	字段含义	备注
partition_name	分区名称	misc和uboot固定，其他可自定义
path	镜像路径	可用相对路径
flash_type	flash类型	只选择nand/nor/emmc
fs_type	文件系统类型	ubifs/jiffs2/none等[1]
shart_addr	分区起始地址	>0
length	分区大小	>0

[1]在emmc下无需指定文件系统类型，选择none即可。

2.2. 相关块设备挂载情况

单板正常启动后，可在 /dev/by_name 路径下查找到相关块设备，如下图2-3所示。

图 2-3 相关块设备挂载情况spi nand单独生成升级包流程图

```
~ # ls /dev/by-name -l
total 0
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 bl31_a -> /dev/mtdblock4
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 bl31_b -> /dev/mtdblock5
lrwxrwxrwx    1 root     0              15 Jan  1 00:00 data -> /dev/mtdblock12
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 kernel_a -> /dev/mtdblock6
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 kernel_b -> /dev/mtdblock7
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 misc -> /dev/mtdblock1
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 otfl -> /dev/mtdblock0
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 rootfs -> /dev/mtdblock8
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 rootfs_a -> /dev/mtdblock8
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 rootfs_b -> /dev/mtdblock9
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 uboot_a -> /dev/mtdblock2
lrwxrwxrwx    1 root     0              14 Jan  1 00:00 uboot_b -> /dev/mtdblock3
lrwxrwxrwx    1 root     0              15 Jan  1 00:00 userfs -> /dev/mtdblock10
lrwxrwxrwx    1 root     0              15 Jan  1 00:00 userfs_a -> /dev/mtdblock10
lrwxrwxrwx    1 root     0              15 Jan  1 00:00 userfs_b -> /dev/mtdblock11
```

该路径下块设备分别表示相关镜像所在位置，如bl31_a通过软连接指向/dev/mtdblock4表示分区所对应的块设备。而rootfs、ubootfs、userfs等无“_a”或“_b”后缀的软连接则表示当前slot所用的分区。

2.3. BootCtrl API

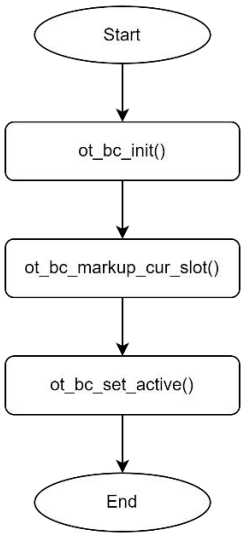
SDK中提供了一个BootCtrl模块，可用于：

1. AB分区升级时修改A/B slot的启动状态信息；
2. 获取system防回滚ID；
3. 设置system防回滚ID；
4. 判断安全启动是否使能；
5. 获取安全校验算法类型。

升级时修改A/B slot的启动状态信息API的调用流程如下图2-4所示，更多具体API详细介绍请参考《SD3589CV100 BootCtrl开发指南.pdf》。

版权所有 © 上海云集互联有限公司

图 2-4 升级时BooCtrl 修改A/B slot的启动状态信息API调用流程



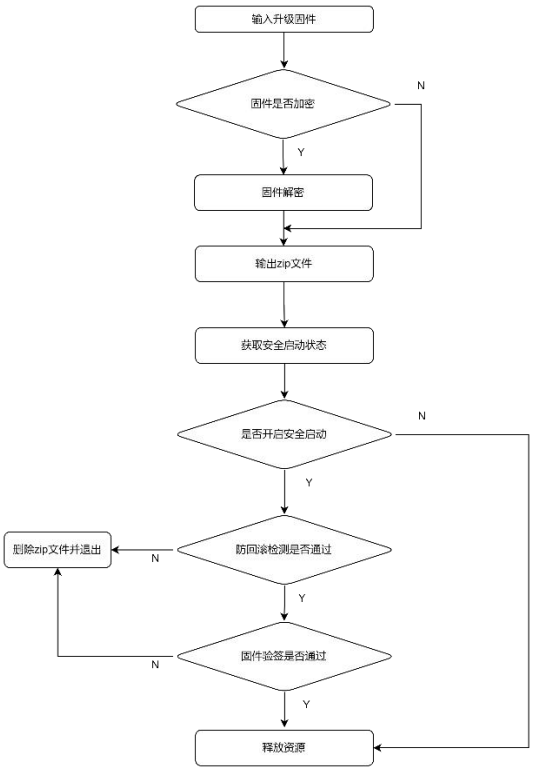
2.4. 防回滚机制

安全启动的情况下，进行升级会对升级的固件做版本对比，只有固件的版本不小于板端的system防回滚ID，才可以进行升级。升级成功后，如果固件版本大于板端的system防回滚ID，将会更新板端的system防回滚ID。如何配置升级固件版本和烧录system防回滚ID详细介绍请参考《SD3589CV100 安全启动开发指南.pdf》。

2.5. 检验与解密

安全启动的情况下，进行升级会对升级的固件进行校验，只有校验通过的固件才可进行升级，其检验过程如图2-5所示。

图 2-5校验升级固件流程图



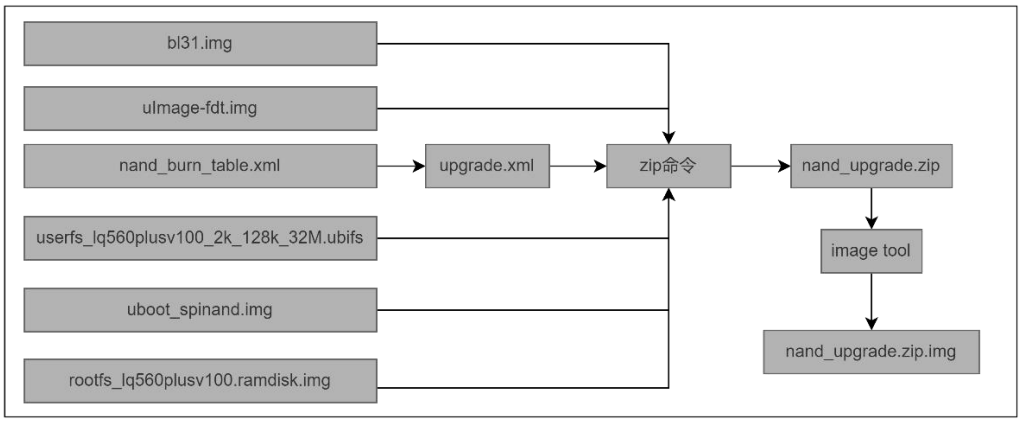
版权所有 © 上海云集互联有限公司

2.6. Upgrade Sample

2.6.1. 升级包的构建

SDK在整编时，默认会构建一个升级Sample程序所使用的升级包文件，若无该升级包，可切换至sdk_root/main/src/plarform/路径下执行 make upgarde单独构建该升级包。其构建过程如图2-6所示。

图 2- 6 spi nand单独生成升级包流程图

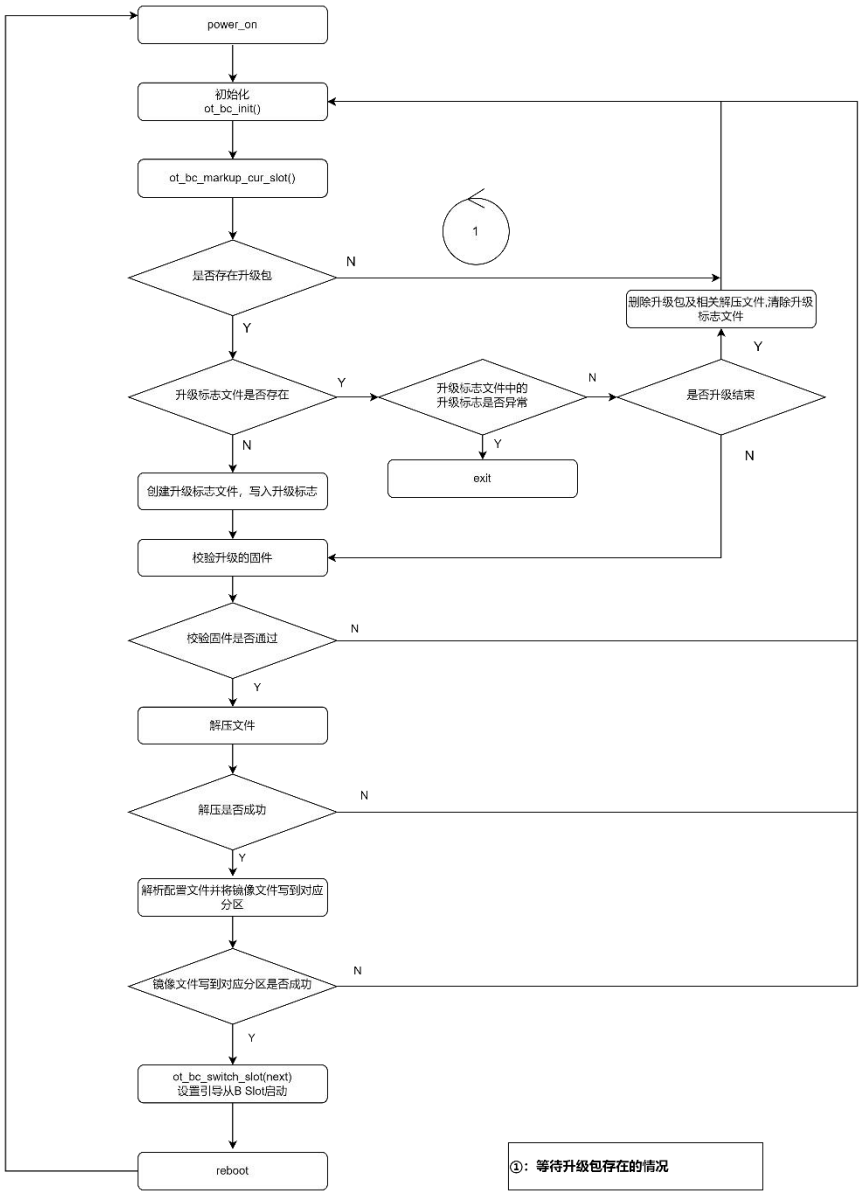


2.6.2. 基于AB分区升级流程

SDK中提供的升级sample程序，目的是为了展示BootCtrl模块API在做AB分区升级时的调用流程，这里以SPI-Nand启动介质为例来介绍升级sample程序（SD3589CV100开发板上的SPI-Nand Flash容量为128MB，升级镜像是被放在了板载的eMMC上，启动时默认将eMMC挂载在/tmp/emmc目录位置，升级时需要将升级包放在该目录下的upgrade目录下），图2-7所示从A slot升级到B slot过程。

版权所有 © 上海云集互联有限公司

图 2-7 升级sample程序的工作流程



升级的具体流程如下：

1. 单板上电，默认从A Slot分区启动；

2. 执行ot_bc_init()接口完成BootCtrl模块初始化(多次调用只会初始一次)；

3. 执行ot_bc_markup_cur_slot()标记当前A slot启动成功，注意该接口每次启动都要调用（多次调用只会标记一次）；

4. 判断是否存在升级包（sample中使用的是/tmp/emmc/upgrade/upgrade.zip.img），若不存在持续执行循环1直至存在升级包；

5. 判断当前升级标志文件是否存在，由于当前是第一次升级，该标志文件不存在，跳转至步骤8；如果存在，跳转至步骤6；

6. 判断升级标志文件中的升级标志是否异常，如果异常则退出程序；

7. 判断当前是否升级结束（sample中的接口为is_upgrade_done），由于第一次升级失败，此时升级未结束，跳转至步骤9；

8. 创建升级标志文件，写入升级标志（sample中的接口为set_upgrade_flag(next)）；

版权所有 © 上海云集互联有限公司

9. 校验升级的固件，具体流程可以参考检验与解密章节（sample中的接口为upgrade_verify()）；

10. 判断校验结果；如果失败，跳转至步骤2；

11. 解压文件（sample中的接口为sample_upgrade_process_package()）；

12. 判断解压文件是否成功；如果失败，跳转至步骤2；

13. 解析配置文件并将镜像文件写到对应分区（sample中的接口为upgrade_parts()）；

14. 判断镜像文件写到对应分区是否成功；如果失败，跳转至步骤2；

15. 执行ot_bc_switch_slot(next)接口设置引导从B Slot区启动；

16. 单板从B Slot分区重启；

17. 执行ot_bc_init()接口完成升级初始化；

18. 执行ot_bc_markup_cur_slot()接口标记当前升级分区成功

19. 判断是否存在升级包（sample中使用的是/tmp/emmc/upgrade/upgrade.zip.img），由于第一次升级后升级包未被删除，此时升级包存在；

20. 判断当前升级标志文件是否存在，由于第一次升级后标志文件未被清除，该标志文件存在；

21. 判断升级标志文件中的升级标志是否异常，如果异常则退出程序；

22. 判断当前是否升级结束（sample中的接口为is_upgrade_done），此时升级已完成, 根据情况设置system防回滚ID；

23. 删除升级包及相关解压文件，清除升级标志文件；

24. 跳转至步骤2并执行循环1。

版权所有 © 上海云集互联有限公司